

VeroFlow — Complete Learning Resources (Mapped to Your Build Plan)

Organized in the exact order you're building: **APK + Instruction → NLP → ChromaDB RAG → Appium (Android) execution + screenshots → Jenkins (added last)**. Plus long-term resources for the SaaS/market-competitive goal.

STAGE 1: Python & FastAPI (Your Backend Foundation)

Resource	Link/Search
Official FastAPI Docs (best starting point)	https://fastapi.tiangolo.com/tutorial/
Python official docs	https://docs.python.org/3/tutorial/
YouTube search	"FastAPI crash course 2025" or "2026"
YouTube search	"Python for beginners full course" — freeCodeCamp

What to focus on: Path/query parameters, request bodies (Pydantic models), file uploads (`UploadFile`), async endpoints. You already have working code — use these to understand *why* it works, not just copy-paste.

STAGE 2: NLP Parsing + Ollama (Local LLMs)

Resource	Link/Search
Ollama official docs	https://github.com/ollama/ollama/blob/main/docs/README.md
Ollama Python library docs	https://github.com/ollama/ollama-python
YouTube search	"Ollama tutorial run local LLM 2026" (filter by upload date — this tool updates fast)
Prompt Engineering Guide (free, comprehensive)	https://www.promptingguide.ai/
YouTube search	"prompt engineering for developers"

What to focus on: How to structure prompts so the model reliably returns clean JSON (this is literally what your Phase 1 NLP parser depends on). Look up "structured output prompting"

and "JSON mode LLM" specifically.

STAGE 3: Embeddings, CLIP & Vector Databases (ChromaDB RAG)

This is the most unfamiliar territory for most students — spend real time here.

Resource	Link/Search
ChromaDB official docs	https://docs.trychroma.com/
OpenCLIP GitHub (the library you're using)	https://github.com/mlfoundations/open_clip
"Vector embeddings explained"	YouTube search — watch 2-3 different explainer videos, this concept clicks better after seeing it explained a few different ways
"What is RAG (Retrieval Augmented Generation)"	YouTube search
"CLIP model explained"	YouTube search — look for ones with visual diagrams, not just talking
Pinecone's learning center (great free articles, vendor-neutral concepts)	https://www.pinecone.io/learn/

What to focus on:

1. What a "vector" actually represents (don't skip this — it's the core idea behind Phase 4-6)
2. Cosine similarity (the math behind "how similar are two things")
3. Why RAG exists — the core idea: "give the AI relevant past examples instead of making it guess from scratch every time"

Suggested order: Watch "vector embeddings explained" FIRST, then "what is RAG", then CLIP-specific content. Trying CLIP before understanding embeddings generally will be confusing.

STAGE 4: Appium + ADB (Android Automation — Your Core Execution Engine)

Resource	Link/Search
Appium official docs	https://appium.io/docs/en/latest/
Appium Python Client docs	https://github.com/appium/python-client
Test Automation University (FREE courses, made for exactly this)	https://testautomationu.applitools.com/ — look for their Appium course specifically
YouTube search	"Appium Python tutorial Android beginners"
ADB command reference (official Android docs)	https://developer.android.com/tools/adb
YouTube search	"ADB commands tutorial"

What to focus on: `find_element` strategies (ID, XPath, accessibility ID — know the difference and priority order), `UiAutomator2Options`, taking screenshots, and reading Appium's error messages (you'll see a lot of `NoSuchElementException` — learn to read what it's telling you).

Bonus tip: Use Appium Inspector (already installed per our last conversation) alongside these tutorials — inspecting a real app's elements while learning theory makes it click much faster than theory alone.

STAGE 5: Jenkins (CI/CD — Added Last, As You Planned)

Good call saving this for last — no point learning it until the core pipeline works.

Resource	Link/Search
Jenkins official docs	https://www.jenkins.io/doc/
Jenkins Pipeline tutorial (official)	https://www.jenkins.io/doc/book/pipeline/
YouTube search	"Jenkins tutorial for beginners 2025/2026"
YouTube search	"Jenkins pipeline Android build tutorial"

What to focus on: Freestyle jobs first (simpler), then Jenkinsfile/Pipeline syntax once comfortable. You'll need Jenkins to trigger your FastAPI backend via webhook/API call — that connection matters more than mastering every Jenkins feature.

SUPPORTING TOOLS: Docker & PostgreSQL

Resource	Link/Search
Docker official "Get Started"	https://docs.docker.com/get-started/
YouTube search	"Docker for beginners crash course"
PostgreSQL official tutorial	https://www.postgresql.org/docs/current/tutorial.html
YouTube search	"PostgreSQL tutorial for beginners"

STAGE 6 (Later): Next.js Dashboard

Not urgent yet since your HTML dashboard works as an MVP — learn this once the pipeline is functionally complete.

Resource	Link/Search
Next.js official "Learn" course (free, interactive)	https://nextjs.org/learn
YouTube search	"Next.js tutorial for beginners"

LONG-TERM: SaaS & Market-Competitive Skills (Post-FYP)

Since your real goal extends beyond FYP into a SaaS product and staying competitive in the market:

Resource	Why
"The Mom Test" by Rob Fitzpatrick (book)	How to validate a SaaS idea by talking to real potential customers — extremely practical, short read
Indie Hackers (community + articles)	https://www.indiehackers.com/ — real founders sharing how they built and priced SaaS products
Y Combinator's Startup School (free)	https://www.startupschool.org/ — structured, free course on building a startup from scratch
YouTube search	"how to price a SaaS product"
YouTube search	"SaaS MVP to launch case study"
Stripe's guide to SaaS billing (practical, once you're ready to charge money)	https://stripe.com/guides/atlas/saas-pricing

Suggested approach: Don't dive deep into these until FYP-II is submitted — but skim "The Mom Test" now if you can, since the mindset (validate before building more) is useful even while finishing the FYP.

SUGGESTED LEARNING ORDER (Matches Your Build Plan)

Week-by-week as you build:

While building Phase 1 (NLP) → Learn: Ollama + Prompt Engineering

While building Phase 2 (Appium) → Learn: Appium + ADB (Test Automation University)

While building Phase 4/6 (CLIP+RAG) → Learn: Embeddings → RAG → CLIP (in that order)

While building Jenkins (last) → Learn: Jenkins basics + Pipeline syntax

Ongoing, in background → Skim SaaS resources when you have spare time

One honest note: You don't need to master any of these fully before touching the code — the usual real pattern is: try the code first (you already have working snippets), hit an error or confusion, THEN watch the specific tutorial that explains that exact piece. Trying to fully learn every topic upfront before writing code will slow you down more than it helps.